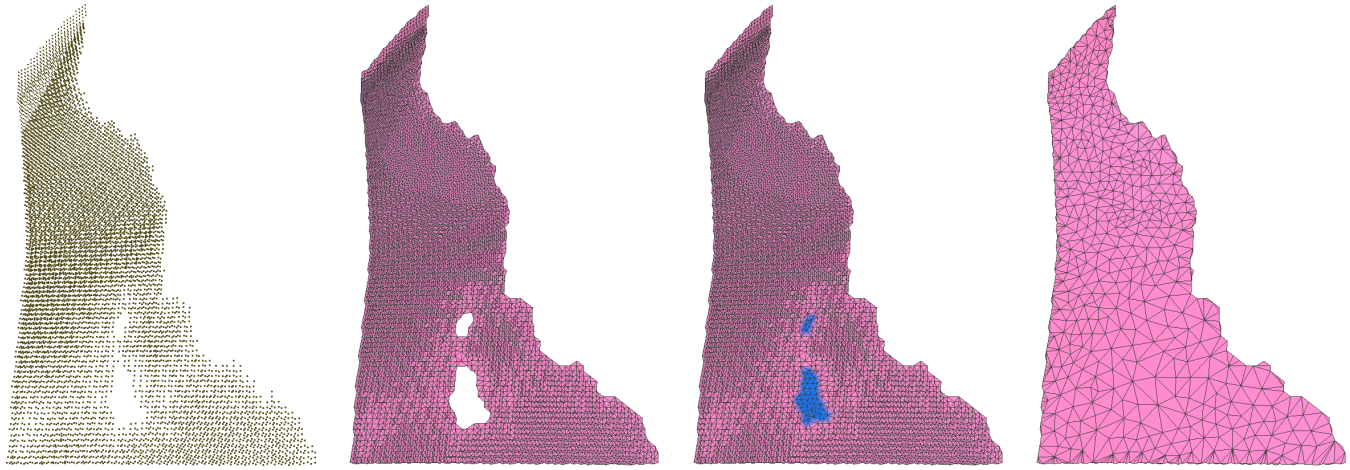
ScrollFiesta: A Virtual Meshing and Unwrapping Tool for the Herculaneum Papyri



(a) Point cloud (b) Ball-pivoting mesh (c) Hole filling (d) Simplification

Fig. 1. Overview of the ScrollFiesta per-cube meshing pipeline, shown on a single chart. (a) A point cloud is extracted from the voxel data, one point per solid voxel, and smoothed onto the sheet. (b) The ball-pivoting algorithm meshes the cloud into a single-sided surface, leaving small holes (white) where the advancing front could not close. (c) Interior holes are detected and filled, with the inserted patches highlighted in blue. (d) The surface is decimated by QEM simplification, giving the final clean chart.

Reading the carbonized Herculaneum papyrus scrolls requires *virtually unwrapping* them - flattening each rolled sheet to a plane so its ink can be read - which in turn requires a clean triangle mesh of the sheet. Producing that mesh has remained a bottleneck. The best automatic input is a per-voxel segmentation from a U-Net classifier, but converting this prediction into a usable mesh is difficult, and the prediction is itself imperfect, leaving holes and *bridges* that erroneously fuse adjacent wraps. We observe that this meshing problem is fundamentally unlike classical surface reconstruction: a scroll sheet is a *single-sided*, open surface, whereas methods such as marching cubes and Poisson reconstruction are built to produce two-sided, watertight ones. We introduce *ScrollFiesta*, a pipeline that produces single-sided, consistently-oriented sheet meshes by projecting the segmented voxels onto a thin point cloud and triangulating it with ball-pivoting. Because every stage is local, the pipeline runs independently on each voxel cube and welds the per-cube meshes into arbitrarily large surfaces, scaling toward whole scrolls. The mesh also exposes the prediction's topological errors directly, letting us separate erroneously-bridged wraps and fill holes. We present preliminary results on dense scroll regions and position ScrollFiesta as the meshing front-end of a complete virtual-unwrapping system.

Author's Contact Information:

ACM XXXX-XXXX/2026/6-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

ACM Reference Format:
. 2026. ScrollFiesta: A Virtual Meshing and Unwrapping Tool for the Herculaneum Papyri. 1, 1 (June 2026), 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

A vast library of papyrus scrolls in ancient Herculaneum was buried beneath volcanic mud and ash during the 79 AD eruption of Mount Vesuvius. These scrolls are carbonized, but remarkably well-preserved, and high-quality synchrotron micro-CT voxel data of the scrolls exists. It has also been shown that it is possible to read ink directly from these scrolls, given a method to *virtually unwrap* them. This virtual unwrapping finds a correspondence between the scroll geometry, embedded in 3D space, and a 2D flat plane; in practice, this means producing a manifold, orientable, approximately developable triangle mesh embedded in the voxel data of the scroll to be unwrapped, and then parameterizing it using an off-the-shelf algorithm such as ABF++[Sheffer et al. 2005]. We introduce *ScrollFiesta*, an end-to-end pipeline that takes segmented voxel data as input and produces mesh geometry satisfying the requirements for virtual unwrapping.

Most virtual unwrapping work has been done by hand, using teams of manual annotators. The best known automatic solution converts raw CT voxel data to a segmentation map using a U-net based prediction algorithm [Ronneberger et al. 2015], but it is unclear how to convert this segmentation map to a useful triangle mesh. Additionally, the U-net prediction is not perfect and may introduce holes or other artifacts, such as artificially joining sections of the

scroll; these so-called *bridges* prevent the scroll from being flattened. Previous work on improving the neural network U-Net predictions has largely focused on tuning the machine learning mechanisms used, as opposed to trying to take the raw predictions and fixing them directly via post-processing. ScrollFiesta produces an end-to-end pipeline for meshing scrolls that takes annotated voxel data as input, either generated by manual annotation or U-Net, and outputs clean, single-sided triangle geometry.

Virtual unwrapping of scroll geometry faces several problems that are not handled by existing meshing methods. First, most algorithms for recovering surface data from voxel inputs (e.g Poisson Surface Reconstruction [Kazhdan et al. 2006] or marching cubes [Lorensen and Cline 1998]) assume the input is two-sided, and attempt to reconstruct it as such. In contrast, virtual unwrapping of scrolls requires the construction of a single-sided sheet that should be manifold, genus-0, and as developable as possible. The orientation of the scroll produced must be consistent, with normal orientation of triangles consistently pointing either inwards or outwards towards the scroll's center. Finally, it is desirable that a virtual unwrapping method must be able to operate on portions of the scrolls being unwrapped, as well as the entire scroll, as the scroll data is very large.

We address these challenges by applying a targeted meshing approach, in which sheet geometry is first flattened and cleaned using a moving least-squares projection operator [Levin 2003] to fit point cloud geometry to flat sheets. These sheets are then separated, aligned to have consistent normal orientation, and meshed using a ball-pivoting algorithm [Bernardini et al. 1999], which naturally produces the flat sheets we seek. ScrollFiesta generates output triangle meshes for individual voxel cubes, and can then weld them by extending the same ball-pivoting algorithm to scroll sheet fronts to connect them in a natural way.

A key advantage of our approach is that using a local projection and local meshing operation ensures that the per-cube meshes we produce remain consistent across cube boundaries. This enables us to weld individual cube pieces together to assemble the complete geometry for the entire scroll being unwrapped. ScrollFiesta is therefore scalable, operating on individual cubes and then combining per-cube solutions into newer, larger meshes, making it suitable for operation on very dense sets of CT information such as 1.2um scans. It has a built-in level of detail mechanism, allowing output meshes to be investigated in meshlab and parameterized and operated on in real time. Most critically, because it reliably produces single sided meshes, it can correct the topological defects in the original scroll geometry, providing a path forwards to fully automatic virtual unwrapping. This includes separating sheets with *bridge material* between them, formed either by failures of the U-Net module or by the volcano itself deforming geometry into hard-to-recover configurations; and detecting and filling holes in the original NN-unet predictions.

In summary, we make the following contributions. First, we identify scroll meshing as a *single-sided* surface-reconstruction problem, and show that projecting the segmented voxels to a sheet and triangulating with ball-pivoting solves it directly, whereas adapting a two-sided reconstruction algorithm does not. Second, we make every stage of this pipeline local, so that per-cube meshes are consistent across cube faces and can be stitched into arbitrarily large surfaces by a divide-and-conquer ball-pivoting weld. Third, we use the resulting mesh to repair the topological errors in the underlying U-Net prediction, separating erroneously-bridged wraps and filling holes. We validate these contributions on dense regions of real Herculaneum scroll data.

This working paper represents a preliminary "progress report" submission for interested parties.

2 Related Work

Virtual unwrapping of the Herculaneum scrolls. Virtual unwrapping was introduced by Seales et al. [2016], who recovered text from the En-Gedi scroll by segmenting its surface within a CT volume and flattening it to a plane. Applying the same idea to the far more tightly-wound Herculaneum scrolls has driven several recent automatic approaches, collected by the Vesuvius Challenge. *Spiral-fitting* [Henderson 2026] uses the fact that a scroll is, in its original form, a single rolled sheet, and fits an extruded spiral to the prediction by solving a variational problem; this is elegant, but degrades wherever the neural-network prediction is too noisy for any single spiral to fit. *Surface-tracing* approaches find small patches on the surface prediction and grow them iteratively, turning the problem into an advancing front. To the best of our knowledge, the open difficulty with this approach is how adjacent patches are merged and how the resulting topological artifacts are resolved, especially when the merge is performed after flattening the patches to two dimensions. *Thaumato Anakalyptor* extracts a point cloud using blurring and Sobel edge detection, classifies the points into wraps, and reconstructs the surface by solving a winding-angle assignment problem, sharing the prediction-noise sensitivity of spiral-fitting. ScrollFiesta also grows an advancing front via ball-pivoting, but handles topology directly on the reconstructed mesh rather than in prediction or winding-angle space. In addition to avoiding the necessity of complicated front merges, this in-situ approach makes topological repair such as hole filling and bridge splitting feasible.

Surface reconstruction and meshing. Surface reconstruction is a mature field, but its standard tools are a poor fit here because they are built to produce *two-sided, watertight* surfaces. Marching cubes [Lorensen and Cline 1998] and its descendants, such as dual contouring [Ju et al. 2002] and the differentiable FlexiCubes [Shen et al. 2023], extract a closed isosurface; Poisson reconstruction [Kazhdan et al. 2006] fits a closed indicator function and, in its distributed form [Kazhdan and Hoppe 2023], scales to large volumes. The main problem with these classical approaches is that a scroll sheet is an open, single-sided manifold: applying methods designed for closed surfaces yield double-sided envelopes that must subsequently be converted to single-sided sheets. Before designing our current approach, we investigated the possibility of meshing the Herculaneum papyri using both marching cubes and Poisson surface reconstruction as starting points. These failures motivated our current algorithm.

3 Algorithm Overview

Our algorithm takes as input one or more volumetric cubes of voxel data, where input voxels are defined as being either *solid* or *empty*. These voxel cubes can be generated by manual annotation, or by neural network classification methods. The output of ScrollFiesta is a triangle mesh of the scroll, connected across cube boundaries, and with repair operations applied to the scroll geometry.

We therefore design a two-part local meshing operation that can operate on individual cubes or components of cubes, while also being able to weld across cube boundaries (Fig. 1). We first synthesize raw point clouds from the input voxel data, flatten them to a smooth sheet, and assign point clouds correct orientations and normals. We then mesh each of these sheets using a classical advancing-front algorithm [Bernardini et al. 1999] which, by design, can also be extended across cube boundaries. Following this, we perform topological cleanup operations as desired on the generated sheets using the initial surface as a prediction; this includes filling holes in the original NN prediction and performing mesh simplification. A subsequent downstream step then welds cubes together using the same process. If additional holes are detected, they can be filled during this time.

4 Per-Cube Mesh Extraction

We initialize our point cloud by extracting one point at the centre of each solid voxel of the input cube. Where the prediction is more than one voxel thick this yields a slab of points several voxels deep, so we first collapse and denoise it into a single clean sheet using an iterated moving least-squares (MLS) projection [Levin 2003]: each point is moved along its local surface normal, estimated from a Wendland-weighted neighbourhood [Wendland 1995], onto the local least-squares plane. Only the through-thickness component of this motion is non-zero, so the slab is thinned to a single sheet without the in-plane contraction we observed when we initially tried the closely-related locally optimal projection [Lipman et al. 2007]. The projection itself is a standard tool; our observation is that choosing a projection method whose kernel has compact support makes the per-cube result consistent across cube boundaries, and that this boundary-consistency is precisely what makes our divide-and-conquer scroll meshing possible. Concretely, adjacent cubes are processed with an overlapping halo of shared input voxels, and because each projected position depends only on neighbours within a fixed radius, points in the shared region land in the same place in both cubes; the two cubes therefore present matching geometry to the welder. Finally, we trim the cloud so that the open mesh boundary falls inside the cube face, and weld points lying within 0.25 voxels of one another.

We then assign the point cloud a globally consistent normal orientation using the method of Hoppe et al. [1992], to ensure that all papyrus surfaces are consistently oriented as our algorithm requires.

Meshing the point cloud requires an algorithm that is local (in order to join cubes together after processing); fast; and that is not designed to output a closed, watertight surface. We therefore use the ball-pivoting algorithm [Bernardini et al. 1999] (BPA), which we summarize as follows: three points form a triangle if a ball of a user specified radius ρ touches them without containing any other point. Starting with a seed triangle, the ball pivots around each edge until it touches another point, forming a triangle. The process continues until all reachable edges have been tried, and then starts from another seed triangle until all points have been considered. We found that this simple approach satisfies our main criteria, at the cost of some small topological artifacts which are easily resolved (see below): it can be used to weld neighbouring cube surfaces into the complete scroll by setting the cube boundaries to the advancing front at the algorithm start, and it is very well-behaved on open surfaces. Our implementation of BPA includes a dihedral fold-back guard which prevents the advancing front from folding over itself.

4.1 Topological Cleanup

At this stage, we may have multiple components erroneously attached in the same cube. This can be due either to proximity during the BPA algorithm, or more commonly due to errors in the input. As we now have meshes, we can detect these connections by finding every connected mesh component in the cube, and casting rays from each triangle centroid along its normal in both directions to determine if we should split the input geometry.

We categorize splits into two cases. First, we consider the case where one small bridge connects two levels of geometry; this is commonly caused by scroll NN prediction artifacts (Fig. 2). We formulate finding these splits as a min-cut problem, and we solve it using the standard Edmonds-Karp algorithm. For more complex overlapping geometries, we use a lifted multicut solver [Keuper et al. 2015] to solve an overlap reduction problem based on the formulation from Limper et al. [Limper et al. 2018].

If holes are detected inside a component, we patch them at this stage (Fig. 3). This may include holes caused by the BPA algorithm, which tend to be single isolated triangles and can be dispatched trivially. Larger holes are detected and filled using the method of Liepa et al. [2003]; our current implementation omits Liepa's minimum-weight triangulation step, instead filling each hole by projecting its boundary to a plane and applying a constrained Delaunay triangulation.

After hole filling, a round of mesh simplification and optimization is applied. Finally, the box is trimmed again to ensure that the meshed cube geometry fits inside the cube boundary.

5 Global Cube Welding

Once individual cubes are meshed, they can be joined to mesh more of the scroll. We load all cubes to be merged into memory, find their boundary half edges, and re-initialize the BPA using those boundary half-edges as the initial front. As this operation may now connect sheet boundaries into closed hole loops, we run an additional pass of hole filling in order to close gaps where possible. This process can then be repeated as desired, ideally using a hierarchical approach to match the whole scroll.

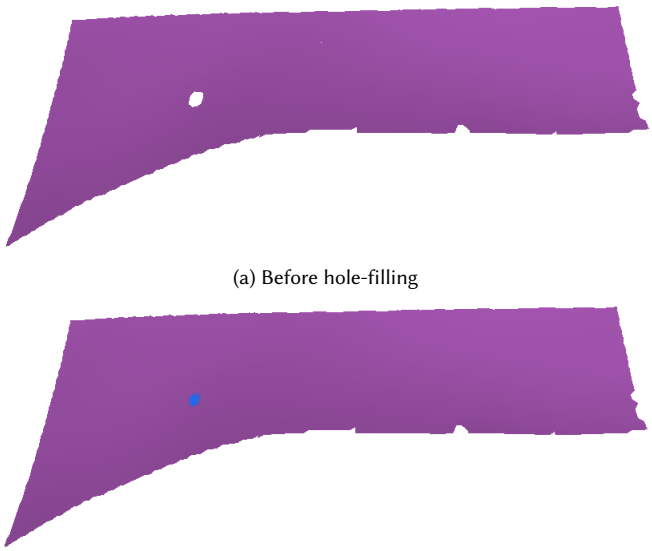
6 Preliminary Results

We stress that these results are preliminary, as this is a progress report and not an official attempt to claim the larger unwrapping prize entry, or a formal submission to a graphics conference.

(a) Before bridge-split

(b) After bridge-split

Fig. 2. Separating an erroneously-bridged component. The U-Net prediction has fused two scroll sheets into a single connected sheet through a thin neck of bridge material; ScrollFiesta detects the bridge and severs it using a min-cut algorithm, recovering the two distinct sheets.



(a) Before hole-filling

(b) After hole-filling

Fig. 3. Hole filling on a single extracted sheet. Holes arise both from the ball-pivoting front and from gaps in the original U-Net prediction; small holes are dispatched trivially and larger loops are closed using the method of Liepa et al. [2003].

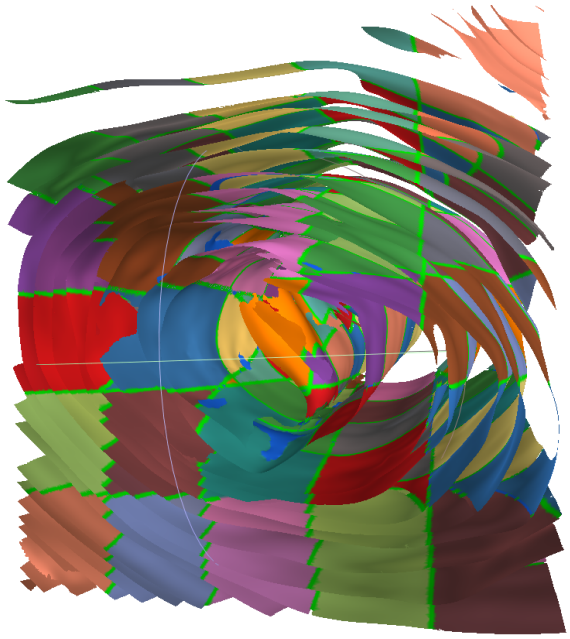


Fig. 4. The welded surface mesh of the central $4 \times 5 \times 5$ cube region (100 cubes of 128^3 voxels) of the *PHerc0139* scroll. Each per-cube chart is drawn in a distinct color; the ball-pivoting seam weld stitches the charts across cube boundaries into the concentric scroll wraps visible at center. The complete mesh has 1.63M vertices and 3.02M triangles.

Welding Across the Grid. Fig. 4 shows our largest test to date: the central $4 \times 5 \times 5$ region of the *PHerc0139* dataset, comprising 100 cubes of 128^3 voxels. The 100 per-cube meshes were extracted concurrently in 415 s of wall-clock time and were then stitched in a single welding pass. The result is a mesh of 1,628,178 vertices and 3,022,017 triangles.

We measure the success of our welding strategy by looking for non-manifold edges or co-oriented foldovers. Across 4,487,120 interior edge-pairs, we have only 76 co-oriented foldovers and 133 non-manifold edges, so fewer than one topological defect per ten

thousand edge pairs. Hole filling closes 2435 of 2443 interior holes exposed by the weld; the eight it leaves unfilled are large, legitimately-open sheet boundary artifacts that our interior hole detector declines to force-close due to not having the correct code to handle topological repair post-fill. We report these counts honestly; they are small, but non-zero, and removing them entirely is our ongoing work.

Per-cube meshing and repair. Fig. 5 shows two meshed cube regions from the recent Kaggle Vesuvius Challenge training set (source IDs 118632705 and 3394433588). These cubes are five to ten times denser than the *PHerc0139* grid cubes, producing raw point clouds of 2.07M and 1.92M points respectively. From each, the ball-pivoting mesher and our overlap separator recover a stack of distinct single-sided sheets (11 and 15 respectively), correctly splitting the layers that the U-Net prediction had erroneously joined. Per-vertex QEM simplification then decimates each mesh roughly thirteen-fold (e.g. 2.07M down to 150K vertices); this both supplies the level-of-detail used for interactive inspection and keeps the assembled global mesh tractable. Hole repair on an individual sheet is shown in Fig. 3.

7 Conclusions

We introduce ScrollFiesta, a new meshing toolkit targeting the Herculaneum Papyri CT scans that makes significant progress towards solving the virtual unwrapping problem. Our method robustly meshes scroll data as single-sided, genus-0 regions, and includes support for basic topological repair functions.

Limitations and Future Work. While our method should scale to cube regions of any size, we have so far validated the full weld only on the central $4 \times 5 \times 5$ region of the *PHerc0139* dataset. Extending it to an entire scroll will require applying the weld hierarchically and repeatedly, rather than in the single flat pass tested here.

Several defects also remain to be eliminated. As previously noted, the welded $4 \times 5 \times 5$ mesh still contains 76 foldovers and 133 non-manifold edges; these are residual geometric overlaps that our winding-repair pass cannot resolve and that the dihedral guard during welding does not retroactively remove. Driving both counts to zero is the most important outstanding correctness task. The dominant performance cost is per-cube extraction, which runs from roughly 0.5 to 1.6 ks on the densest training cubes, as ball-pivoting front growth, MST-based normal orientation, and hole filling are all serial; this is the natural target for future optimization.

A further open problem is evaluation itself. The Vesuvius Challenge defined quantitative scores (a surface Dice coefficient, variation of information, and a topological score) but in our experience these did not reliably capture the properties that actually matter for unwrapping. In particular, a mesh can score well while still containing the inter-wrap mergers or intra-sheet splits that render a region unreadable, and converting the mesh back to voxel data for score metrics is itself a challenging problem. This is why we report raw topological counts rather than a single benchmark number. Designing an evaluation that faithfully measures single-sidedness, wrap separation, and developability, and that correlates with downstream ink readability, is an important open problem in its own right, and one we have not yet solved.

Our current plan of action is as follows. First, finish driving all remaining topological defects to zero and ensuring that all holes in our test cases are correctly closed and filled, and all bridges are repaired. This is largely a matter of hunting bugs and tuning; in particular, our bridge code was originally developed for an earlier, marching-cubes based approach and needs rework in order to function perfectly on all BPA-generated outputs. Second, make mesh construction hierarchical, and expand it to whole scrolls. Finally, the present pipeline stops at a clean single-sided mesh: closing the loop and producing a flattened, ink-readable parameterization via ABF++ [Sheffer et al. 2005] remains future work, but is simple enough engineering given a high quality mesh. Our work would also benefit from tooling which would let an annotator use ScrollFiesta's topological repair and meshing operations in an interactive manner; this could then be used to correct the NN outputs on individual cube pieces by the annotation team, which in turn could be used for improving the U-Net detection apparatus.

References

Fausto Bernardini, Joshua Mittleman, Holly Rushmeier, Cláudio Silva, and Gabriel Taubin. 1999. The ball-pivoting algorithm for surface reconstruction. *IEEE transactions on visualization and computer graphics* 5, 4 (1999), 349–359.

Paul Henderson. 2026. Virtually Unrolling the Herculaneum Papyri by Diffeomorphic Spiral Fitting. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 6401–6411.

Hugues Hoppe, Tony DeRose, Tom Duchamp, John McDonald, and Werner Stuetzle. 1992. Surface reconstruction from unorganized points. In *Proceedings of the 19th annual conference on computer graphics and interactive techniques*. 71–78.

Tao Ju, Frank Losasso, Scott Schaefer, and Joe Warren. 2002. Dual contouring of hermite data. In *Proceedings of the 29th annual conference on Computer graphics and interactive techniques*. 339–346.

Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. 2006. Poisson surface reconstruction. In *Proceedings of the fourth Eurographics symposium on Geometry processing*, Vol. 7.

Michael Kazhdan and Hugues Hoppe. 2023. Distributed poisson surface reconstruction. In *Computer graphics forum*, Vol. 42. Wiley Online Library, e14925.

Margret Keuper, Evgeny Levinkov, Nicolas Bonneel, Guillaume Lavoué, Thomas Brox, and Bjorn Andres. 2015. Efficient decomposition of image and mesh graphs by lifted multicuts. In *Proceedings of the IEEE international conference on computer vision*. 1751–1759.

David Levin. 2003. Mesh-independent surface interpolation. In *Geometric Modeling for Scientific Visualization*, Guido Brunnett, Bernd Hamann, Heinrich Müller, and Lars Linsen (Eds.). Springer, 37–49.

Peter Liepa. 2003. Filling holes in meshes. In *Proceedings of the 2003 Eurographics/ACM SIGGRAPH symposium on Geometry processing*. 200–205.

Max Limper, Nicholas Vining, and Alla Sheffer. 2018. Box cutter: atlas refinement for efficient packing via void elimination. *ACM Trans. Graph.* 37, 4 (2018), 153.

Yaron Lipman, Daniel Cohen-Or, David Levin, and Hillel Tal-Ezer. 2007. Parameterization-free projection for geometry reconstruction. *ACM Transactions on Graphics (ToG)* 26, 3 (2007), 22–es.

William E Lorensen and Harvey E Cline. 1998. Marching cubes: A high resolution 3D surface construction algorithm. In *Seminal graphics: pioneering efforts that shaped the field*. 347–353.

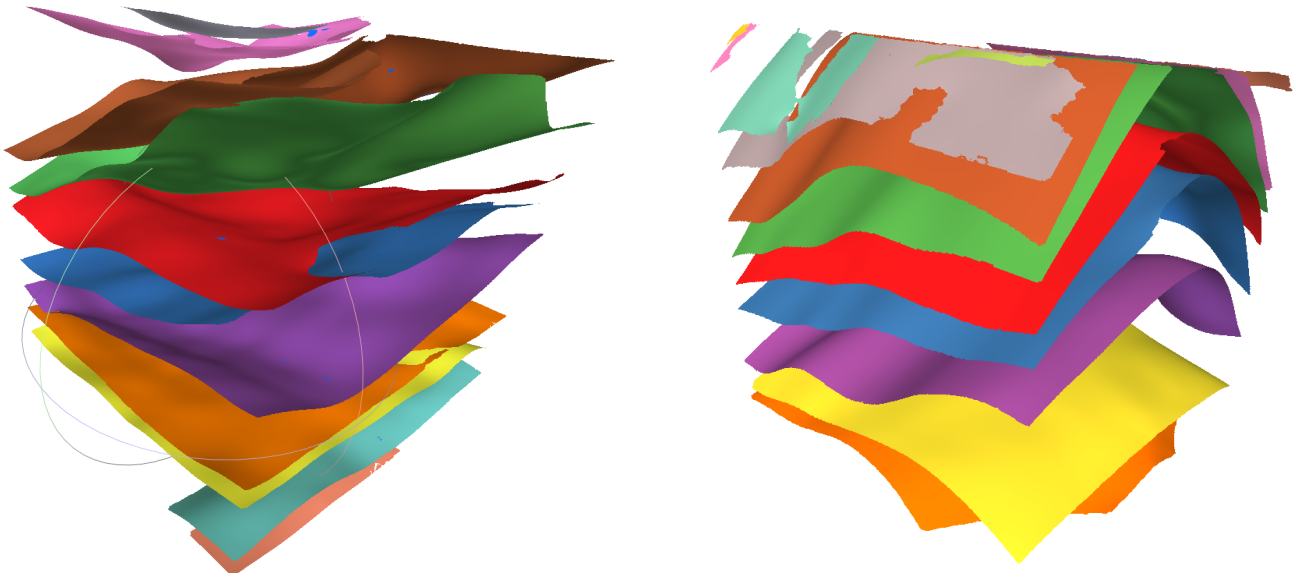
Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*. Springer, 234–241.

William Brent Seales, Clifford Seth Parker, Michael Segal, Emanuel Tov, Pnina Shor, and Yosef Porath. 2016. From damage to discovery via virtual unwrapping: Reading the scroll from En-Gedi. *Science advances* 2, 9 (2016), e1601247.

Alla Sheffer, Bruno Lévy, Maxim Mogilnitsky, and Alexander Bogomyakov. 2005. ABF++: fast and robust angle based flattening. *ACM Transactions on Graphics (ToG)* 24, 2 (2005), 311–330.

Tianchang Shen, Jacob Munkberg, Jon Hasselgren, Kangxue Yin, Zian Wang, Wenzheng Chen, Zan Gojcic, Sanja Fidler, Nicholas Sharp, and Jun Gao. 2023. Flexible isosurface extraction for gradient-based mesh optimization. *ACM Transactions on Graphics (ToG)* 42, 4 (2023), 1–16.

Holger Wendland. 1995. Piecewise polynomial, positive definite and compactly supported radial functions of minimal degree. *Advances in computational Mathematics* 4, 1 (1995), 389–396.



(a) Source cube 118632705 (11 sheets.)

(b) Source cube 3394433588, separated into 15 sheets.

Fig. 5. Two dense cube regions from the Kaggle Vesuvius Challenge training set, meshed and topologically separated by ScrollFiesta. Each raw cube produces a point cloud of roughly two million points; ball-pivoting meshes the cloud and the overlap separator divides it into the individually-colored single-sided sheets shown, recovering the locally-parallel wrap structure where the U-Net prediction had bridged adjacent layers.